CSC 251 Lecture 19: Object-Oriented Programming, Classes, Decorators

Use of Underscores in Python

1) In variable names, snake_case
2) Loop variable placeholder
3) Digit separator, 1_000_000
4) Binding to last lambda function or last computed value
5) Variables
    a. Single leading underscore used for internal variable, they are not imported during * imports
    b. Single trailing underscore to avoid conflict with reserved words (`class`)
    c. Double leading underscores invoke name mangling for non-public variables
    d. Double leading and trailing underscores to define dunder/magic methods

Object-Oriented Programming

Object-Oriented Programming (OOP) is an approach to programming that centers around objects, created from abstract data types that combine data and functions together. Key ideas:

encapsulation: packaging attributes + methods with limited access

data hiding: abstraction, protect data + code

object reusability: use among programs

Example 1: Suppose we have a **clock-type** object. It would contain:

Data attributes: values that define the state of the object.

For clock-type: hour , minutes , seconds time zone

Public methods: functions that can be accessed by entities outside the object

For clock-type: get_time, set_time , set_alarm ...

Private/non-public methods: functions that are part of the object's internal workings, details are hidden

For clock-type: increment_sec , sound_alarm ...

Accessor methods: provide a way for code outside the class to retrieve the attribute values. getters

Mutator methods: allow a data attribute value to be stored or changed. setters

Class Definitions

Example 2a: What do you recognize?

self.side ⇒ public

obj1.sideup

name mangling

__Coin__sideup

class name

capitalized

```
import random

class Coin:
    """
    A class representing a coin.

    Attributes:
        sideup (str): The value of the side facing up, 'Heads' or 'Tails'
    """

    def __init__(self):
        """
        Initializes a new Coin object.
        Args: none
        """
        self.__sideup = 'Heads'

    def toss(self):
        """
        Changes the side facing up based upon random number 0 or 1.
        Args: None
        """
        if random.randint(0, 1) == 0:
            self.__sideup = 'Heads'
        else:
            self.__sideup = 'Tails'

    def get_sideup(self):
        """
        Accessor for the sideup attribute.
        Returns:
            str: The side of the coin facing up
        """
        return self.__sideup
```

pointer to itself, like "this"

Constructor or initializer

non-public

initialized to "Heads"

getter

No setter for this class - Coin must be tossed!

Notes:

- The class name, `Coin`, is capitalized by convention.
- Each of the three methods of the `Coin` class are passed the `self` parameter. Omitting the `self` in front of attribute names creates a local variable.
- The `__init__ method` (that's with 2 underscores on the front and back) listed first is called an initializer method and is automatically executed when an instance of the class is created.
- To make `sideup` a **non-public** variable, include 2 underscores on the front end of the variable: `__sideup`

- Python renames attributes through **name mangling** to prevent naming conflicts during inheritance.
- Class definitions are often stored in modules. The objects are created by specifying the module:

```
import coin
my_coin = coin.Coin()
```

module / file
class instantiated
module

Or import a specific class from the module:

```
from objects import coin
my_coin = Coin()
```

no need to specify

Or import all classes from a module:

```
from objects import *
my_coin = Coin()
```

A good or bad idea?
watch for collisions!

Example 2b: Create an object instance of the Coin class:

```
from coin import *

def main():
    # Create an object from the Coin class.
    my_coin = Coin()
    print('This side is up:', my_coin.get_sideup())

    # Toss the coin 10 times and output the result
    for count in range(10):
        my_coin.toss()
        print(my_coin.get_sideup())

if __name__ == '__main__':
    main()
```

a method for now...

More Pythonic - Access Non-public Attributes with Decorators

There are 2 ways to access non-public attributes:

1) Through the setters and getters
2) Through a decorator method (see https://docs.python.org/3/library/functions.html#property)
   This built-in Python function allows you to access the attributes as **public** variables

   Getters use an `@property` annotation or decorator with a method called `value`

   Setters use an `@value.setter` decorator with a method called `value`

   Deleters use an `@value.deleter` decorator with a method called `value`

Example 2c: Redefine the class with decorators

```
import random

class Coin:
    """
    A class representing a coin.

    Attributes:
        sideup (str): The value of the side facing up, 'Heads' or 'Tails'
    """

    def __init__(self):
        """
        Initializes a new Coin object.
        Args: none
        """
        self.__sideup = 'Heads'

    def toss(self):
        """
        Changes the side facing up based upon random number 0 or 1.
        Args: None
        """
        if random.randint(0, 1) == 0:
            self.__sideup = 'Heads'
        else:
            self.__sideup = 'Tails'

    @property
    def sideup(self):
        """
        Accessor for the sideup attribute.
        Returns:
            str: The side of the coin facing up
        """
        return self.__sideup
```

same

new

more Pythonic, to access as public

Example 2d: Access your attribute as though it was public. Note: the `get_sideup` method no longer works!

```
. . .
    my_coin = Coin()

    print('This side is up:', my_coin.sideup)

    for count in range(10):
        my_coin.toss()
        print(my_coin.sideup)
```

without using getter, as pubic attribute

Plan to use decorators in your Python coding!

- More concise
- Create read-only, write-only, and read-write attributes
- No need to change your user interface when moving from public to private attributes

"Make it easy!"

From PEP8:

"For simple public data attributes, it's best to expose just the attribute name, without complicated accessor/mutator methods. Keep in mind that Python provides an easy path to future enhancement, should you find that a simple data attribute needs to grow functional behavior. In that case, use properties to hide functional implementation behind simple data attribute access syntax."

Example 2e: For multiple instances of the Coin class, each instance has its own set of data attributes, so `self` in the class definition refers to the attributes of that instance.

```
from coin import *

def main():
    # Create three objects from the Coin class.
    coin1 = Coin()
    coin2 = Coin()
    coin3 = Coin()

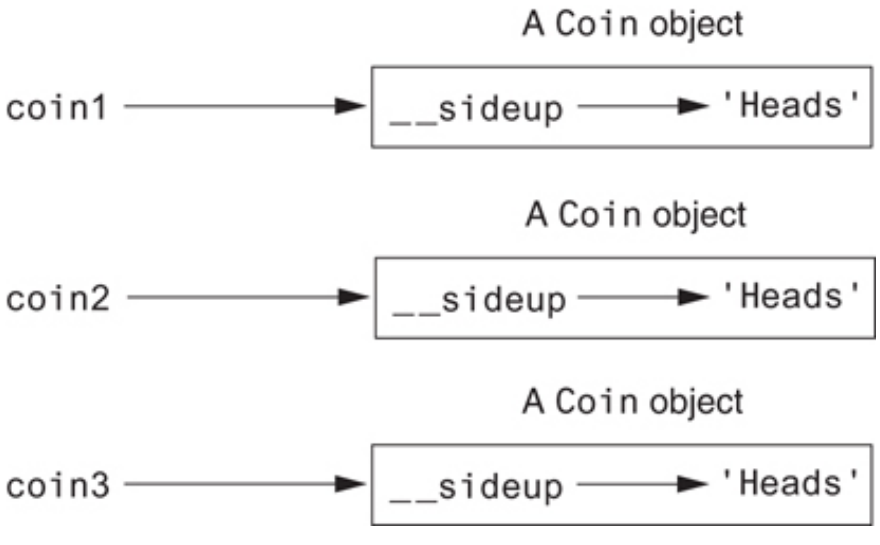
    # Display the side of each coin that is facing up.
    print('Starting with three coins:')
    print(coin1.sideup)
    print(coin2.sideup)
    print(coin3.sideup)
    print()

    # Toss each coin.
    print('Tossing all three coins...')
    print()
    coin1.toss()
    coin2.toss()
    coin3.toss()

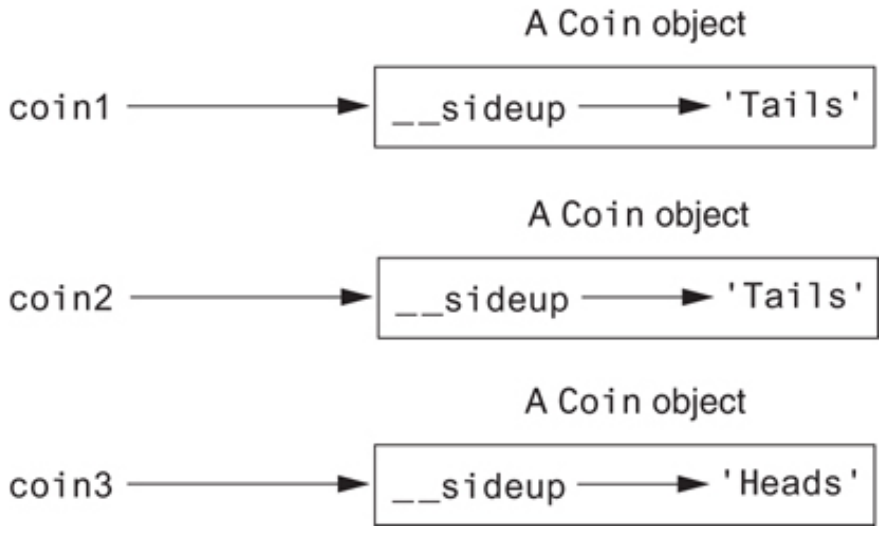
    # Display the side of each coin that is facing up.
    print('After tossing the coins:')
    print(coin1.sideup)
    print(coin2.sideup)
    print(coin3.sideup)
    print()

if __name__ == '__main__':
    main()
```

Upon creation:



After each coin is tossed:



A Class that Accepts Parameters

Example 3a: Define the `BankAccount` class in a file called `bankaccount.py`:

```
class BankAccount:
    """
    A class representing a bank account allowing deposits and withdrawals.
    Attributes:
        balance (float): The current balance of the account
    """

    def __init__(self, bal):
        """
        Initializes a Bank account object.
        Args:
            balance (float): The current amount balance of the account.
        """
        self.__balance = bal

    def deposit(self, amount):
        """
        Increases the balance by the amount deposited.
        Args:
            amount (float): The amount deposited into the account
        """
        self.__balance += amount

def withdraw(self, amount):
    """
    Decreases the balance by the amount withdrawn.
    Args:
        amount (float): The amount withdrawn from the account
    """
    if self.__balance >= amount:
        self.__balance -= amount
    else:
        print('Error: Insufficient funds')

@property
def balance(self):
    """
      Accessor for the balance attribute.
      Returns:
          float: The current balance in the account
    """
    return self.__balance

def __str__(self):
    """
      Str method to format the object's state.
      Returns:
          str: A formatted description of the object's state
    """
    return f'The balance is ${self.__balance:,.2f}'
```

A dunder method that specifies how it should be output

f-string returned for printing

Example 3b: Import the `BankAccount` class in a file named `accounting.py`:

```
import bankaccount

def main():

    start_bal = float(input('Enter your starting balance: '))

    savings = bankaccount.BankAccount(start_bal)

    pay = float(input('How much would you like to deposit into your account? '))
    savings.deposit(pay)

    print(savings)

    cash = float(input('How much would you like to withdraw from your account? '))
    savings.withdraw(cash)

    print(savings)

if __name__ == '__main__':
    main()
```

__str__ method called

When and how is the string method called?

A Class with Default Parameters

Example 3c: Reconsider the `BankAccount` class with default parameters:
Note: documentation omitted but assumed to be present.

```
class BankAccount:

    def __init__(self, bal = 1000):
        self.__balance = bal

    def deposit(self, amount = 0):
        self.__balance += amount

    def withdraw(self, amount = 100):
         . . .

    savings = bankaccount.BankAccount()
    savings.deposit()
    savings.withdraw()
    print(savings)
```

Default values used

Example 4: The following `Product` class includes multiple default parameters. Which can you leave off? What else is new?

Either leave off right-most, to left or use keywords

```
class Product:

    def __init__(self, name = '', price = 0.0, discount = 0):
        self.__name = name
        self.__price = price
        self.__discount = discount

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, n):
        self.__name = n

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, p):
        self.__price = p

    @property
    def discount(self):
        return self.__discount

    @discount.setter
    def discount(self, d):
        self.__discount = d

    def compute_new_price(self):
        return f'$ {self.__price * (1 - self.__discount / 100):,.2f}'

    def __str__(self):
        return f'Product name: {self.__name}, Price: $ {self.__price:,.2f},
           Discount: {self.__discount}%'
```

In program:

```
thing = Product(price=100)
```

default values for name and discount used